Computação Embarcada

Aula 03: Estruturas de repetição e Funções

Parte 1: Revisão

Recapitulando os conceitos da Aula Anterior

Revisão Rápida: Aula 02

Memória

A memória **Flash** armazena o código compilado (sketch). A reduzida **SRAM (2 KB)** é volátil e guarda as variáveis em tempo de execução. Otimização é crucial.

Escopo e Tipos

Variáveis **locais** nascem e morrem dentro das funções (liberando RAM). Variáveis **globais** vivem o tempo todo. Evite usar float devido à falta de FPU em hardware.

Decisão Lógica

O desvio de fluxo baseia-se em if/else e switch/case. Operadores **Bitwise** (&, |, ^, ~) permitem o controle preciso de pinos em registradores de 8 bits.

Parte 2: Repetição

Automação de tarefas contínuas

(Loops)

O Laço for

Contador Definido

Possui três estágios na declaração: inicialização, condição e incremento.

```
for (int i=0; i<10; i++)
```

Ideal quando sabemos exatamente o número de vezes que um bloco de instruções deve ser executado.

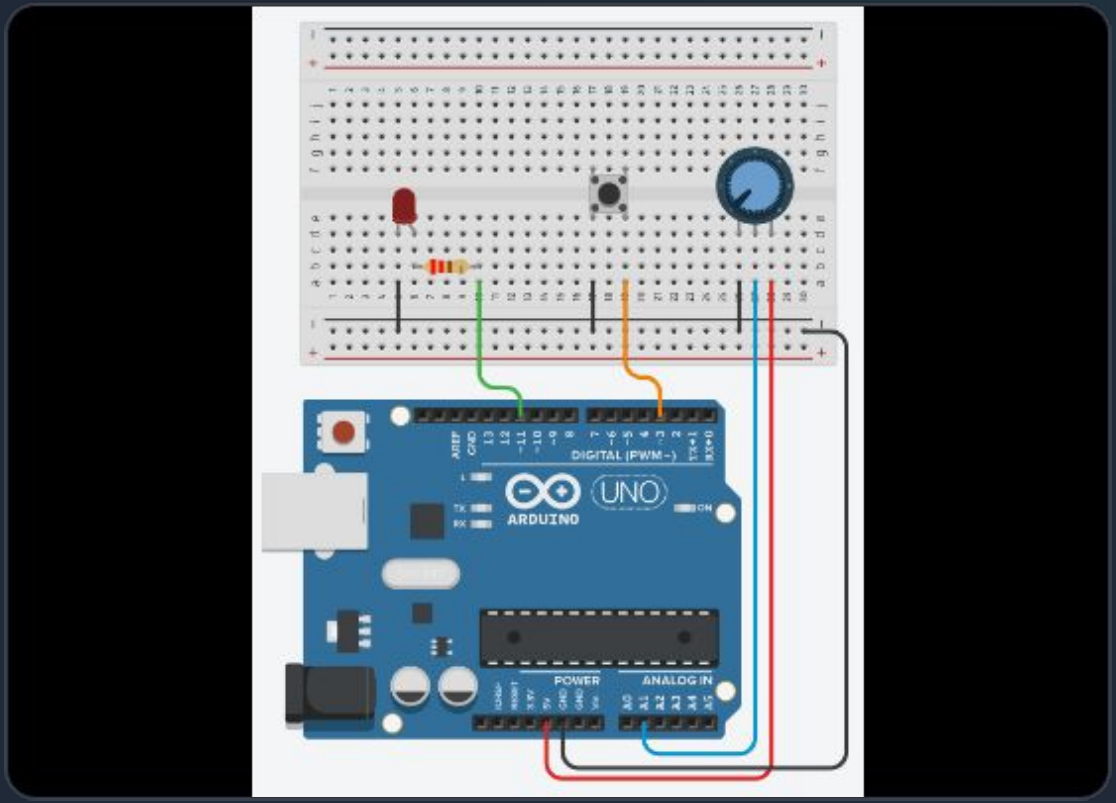
Uso em Embarcados

É a estrutura perfeita para varrer sequências previsíveis. Muito utilizado no Arduino para iterar sobre **Arrays** de sensores, inicializar pinos digitais em lote no setup() e criar padrões luminosos complexos em matrizes de LEDs.

Laços while e do-while

- O while avalia a condição antes de entrar no bloco. Se for falsa logo de cara, o código nunca será executado.

- É ideal para travar a execução do sistema **aguardando um evento físico** externo, como a leitura de um pino conectado a um botão.
- O do-while inverte esta lógica: ele executa as instruções **pelo menos uma vez** antes de testar a condição no final.
- Excelente para inicializar menus interativos em displays ou forçar leituras primárias de sensores antes da validação.



Controle Fino e Aninhamento

break e continue

break: Força a saída imediata do loop ativo, ignorando qualquer iteração restante e transferindo o fluxo para a próxima linha de código externa.

continue: Interrompe apenas a iteração *atual*, pulando direto para a próxima avaliação ou incremento do laço.

Loops Aninhados

Consiste em colocar um loop dentro do bloco de execução de outro loop (ex: um for dentro de um while).

É essencial na eletrônica para varrer dados em **matrizes bidimensionais**, como a leitura de teclados matriciais (linhas x colunas) e multiplexação de displays.

Parte 3: Funções

Modularização em

C/Arduino

Anatomia de uma Função

Tipo de Retorno

Define o que a função devolve ao sistema após terminar a execução. Pode ser um número (int, float), um estado (bool) ou nada (void).

Nome e Assinatura

Identificador único que deve descrever a ação realizada (ex: lerTemperatura). A assinatura inclui o nome e os tipos de dados que ela aceita receber.

Parâmetros

As variáveis de entrada. No Arduino, dados são geralmente passados por **valor** (cópia). Uma função pode exigir múltiplos parâmetros de tipos distintos.

Escopo e a Palavra static

O Risco das Variáveis Globais

Variáveis declaradas no topo do arquivo prendem a escassa SRAM indefinidamente. Além disso, podem ser alteradas acidentalmente por qualquer parte do código (efeitos colaterais invisíveis). Sempre prefira o uso de variáveis locais dentro das funções para liberar memória.

O Modificador static

E se precisarmos que uma variável *local* não seja destruída no fim da função? O uso do static permite que ela "lembre" seu último valor entre sucessivas chamadas, mantendo-se perfeitamente protegida (invisível ao resto do sistema). Útil para criar contadores de estado.

Boas Práticas de Programação

Responsabilidade Única: Uma função deve realizar apenas uma tarefa bem definida. Não misture a leitura complexa de um sensor com a exibição de dados em um display na mesma função.

Nomes Descritivos: Use verbos claros. calcularMedia() e ligarRele() são muito superiores a nomes genéricos como calc() ou funcao1(). O código deve documentar a si mesmo.

Modularidade: Mantenha o seu bloco principal loop() extremamente enxuto e legível. Ele deve atuar apenas como um "maestro", chamando as funções especializadas no momento correto.

Protótipos: Em projetos C++ maiores, declare a assinatura da função antes de tentar usá-la, garantindo que o compilador compreenda a sua estrutura e evite falhas de referência.

Tipos de Retorno Comuns

Tipo de Retorno	Tamanho RAM	Aplicação Prática no Arduino
void	N/A	Executa uma ação sem devolver resposta (ex: ligar um atuador).
int	2 bytes	Devolve números inteiros. Ideal para retornos de analogRead().
bool	1 byte	Retorna Verdadeiro / Falso. Útil para checar o status de um botão.
float	4 bytes	Retorna valores precisos de cálculos. Evite repetições, hardware lento!

Referências Bibliográficas

- **MCROBERTS, M.** Arduino básico. 2. ed. São Paulo: Novatec, 2015. 506 p.
- **MONK, S.** Programação com Arduino: começando com sketches. 2. ed. Porto Alegre: Bookman, 2017.
- **DEITEL, H. M.; DEITEL, P. J.** C: Como programar. São Paulo: Pearson, 2011.
- **SOUZA DE LOPES, J. A.** Lógica Matemática. 1. ed. São Paulo: Pearson, 2016.
- **WEBB, B.; CONSI, T. R.** Biorobotics: methods and applications. MIT Press, 2001.

Hora da Mão na Massa!

Vamos aplicar os conceitos nos exercícios propostos para fixação.